考点3: 数据可视化 - 绘制条形图、饼图、折线图

一.考试重点:

1. Pandas + Matplotlib配合使用
2. 三种基本图表的绘制方法
3. 中文显示问题(可能考)

二.python数据分析使用的模块，举例结合数据表的信息，绘制条形图、饼图、折线图等。

在Python中，进行数据分析通常会使用一些非常流行的库，如Pandas、Matplotlib、Seaborn和Plotly 等。这些库各自有不同的特点和用途：

1. Pandas：用于数据操作和分析，提供了数据结构和数据分析工具，特别是它的DataFrame对象非常 适合处理表格数据。
2. Matplotlib：这是Python中最基础的绘图库之一，支持创建各种静态、动态甚至交互式的可视化图 表。
3. Seaborn：基于Matplotlib构建，提供了一个高层次的接口来绘制统计图形，使得创建复杂的可视化 更加简单。
4. Plotly：一个可以创建交互式图表的库，非常适合用来制作网络应用中的可视化效果。 下面是一些具体的例子，说明如何结合这些库来绘制条形图、饼图和折线图。

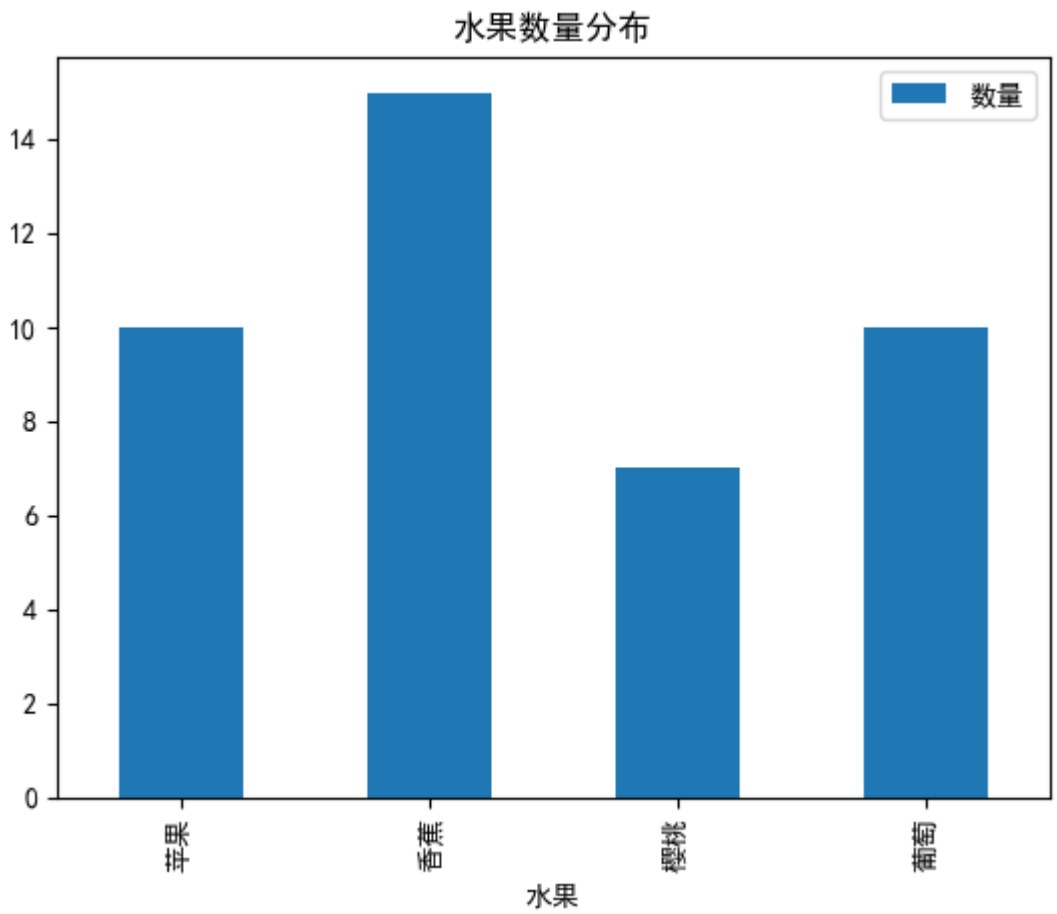
   条形图

```
#导入pandas库处理数据，并给它取了别名pd
import pandas as pd
#导入绘图库，并给它取了个别plt
import matplotlib.pyplot as plt

#处理mac系统中文乱码的问题
#plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']  # Mac
#处理Windows系统中文乱码的问题
plt.rcParams['font.sans-serif'] = ['SimHei']  # Windows
plt.rcParams['axes.unicode_minus'] = False  # 解决负号显示问题


#创建字典类型数据
data = {'水果': ['苹果', '香蕉', '樱桃', '葡萄'],
'数量': [10, 15, 7, 10]}
#数据类型转换：将字典类型的数据转换成表格类型的数据
df = pd.DataFrame(data)
#用条形图展示数据，x轴显示水果这列数据，y轴表示数量这列数据，title表示图形标题
df.plot(kind='bar', x='水果', y='数量', title="水果数量分布")
#显示图表
plt.show()
```

运行结果如下：



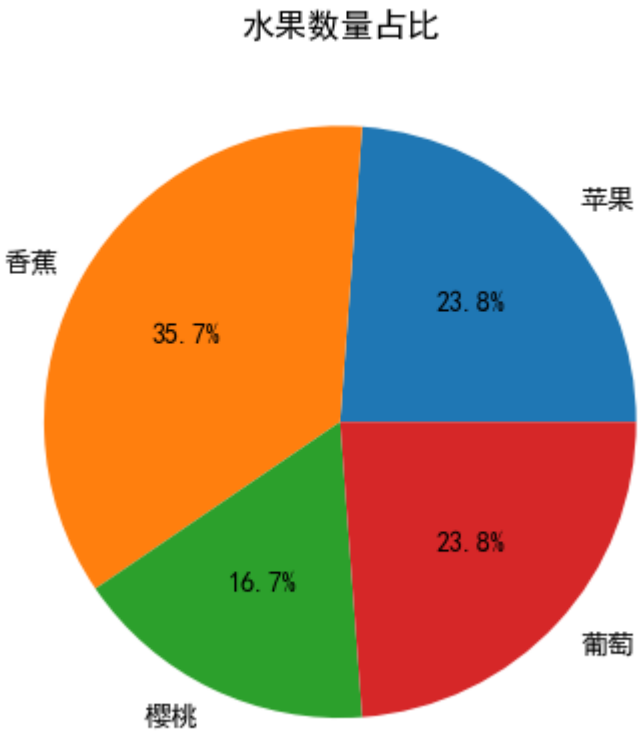
饼图

```
#导入pandas库用于创建和处理表格数据，并取了个别名pd
import pandas as pd
#导入绘图库，并取了个别名plt
import matplotlib.pyplot as plt

#处理mac系统中文乱码的问题
#plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']  # Mac
#处理Windows系统中文乱码的问题
plt.rcParams['font.sans-serif'] = ['SimHei']  # Windows
plt.rcParams['axes.unicode_minus'] = False  # 解决负号显示问题

#创建字典数据
data = {'水果': ['苹果', '香蕉', '樱桃', '葡萄'],
'数量': [10, 15, 7, 10]}
#数据类型转换，将字典类型的数据转换为dataframe表格类型数据
df = pd.DataFrame(data)
#绘制饼图，调出数量列作为扇形区域的大小，水果名称列作为扇形区域的标签名，autopct表示百分比
plt.pie(df['数量'], labels=df['水果'], autopct='%1.1f%%')
#设置图形标题
plt.title('水果数量占比')
#显示图形
plt.show()
```

运行效果如下：



折线图

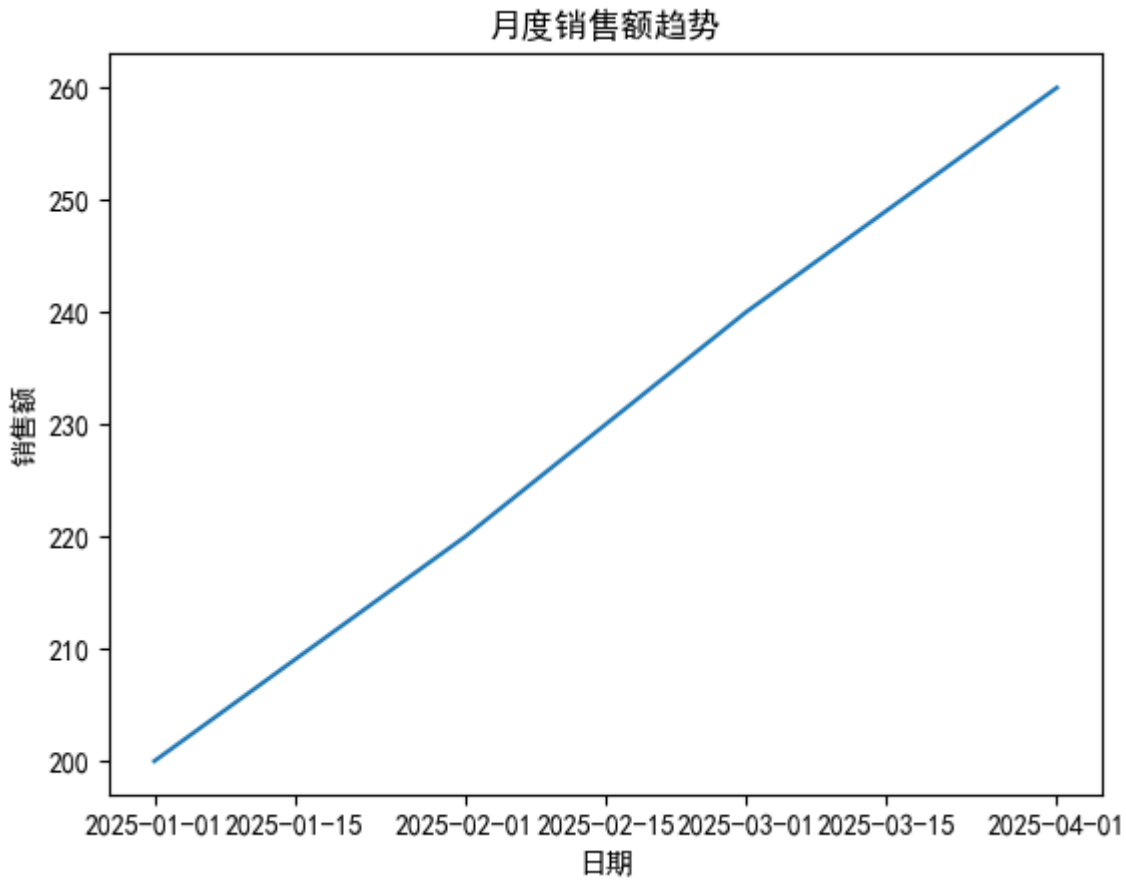
```
#导入pandas库创建和处理表格数据,并取了个别名pd
import pandas as pd
#导入绘图库，并取了个别名plt
import matplotlib.pyplot as plt

#处理mac系统中文乱码的问题
#plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']  # Mac
#处理Windows系统中文乱码的问题
plt.rcParams['font.sans-serif'] = ['SimHei']  # Windows
plt.rcParams['axes.unicode_minus'] = False  # 解决负号显示问题

#创建字典类型的数据
data = {'日期': ['2025-01-01', '2025-02-01', '2025-03-01', '2025-04-01'],
'销售额': [200, 220, 240, 260]}
#数据类型转换，将字典类型的数据转换为padas中dataframe数据
df = pd.DataFrame(data)
#数据类型转换：将日期这列的字符串类型的数据转换为datetime日期类型的数据
df['日期'] = pd.to_datetime(df['日期'])
#绘制拆线图，将日期这列作显x轴，将销售额作为y轴
plt.plot(df['日期'], df['销售额'])
#图形标题
plt.title('月度销售额趋势')
```

```
#设置x轴标签
plt.xlabel('日期')
#设置y轴标签
plt.ylabel('销售额')
#显示图形
plt.show()
```

运行效果如下：



三.案例

1.案例一

```
#导入创建和处理表格数据的库，并取了个别名叫pd
import pandas as pd
#导入绘图库，并取了个别名叫plt
import matplotlib.pyplot as plt
#导入数值计算的库，并取了个别名np
import numpy as np

#解决中文显示问题
#解决mac系统中文乱码问题
#plt.rcParams['font.sans-serif'] = ['Arial Unicode MS'] # Mac
#解决Windows系统中文乱码问题

plt.rcParams['font.sans-serif'] = ['SimHei'] # Windows
```

```python
#解决负号显示问题
plt.rcParams['axes.unicode_minus'] = False

#定义绘制条形图的函数
def plot_bar_chart():
    """
    条形图 - 用于展示类别间的数值对比
    考试常见：各科成绩对比、销售额对比等
    """
    #输出提示
    print("="*60)
    print("1. 条形图绘制")
    print("="*60)

    #创建一个字典类型数据
    data = {
    '水果': ['苹果', '香蕉', '樱桃', '葡萄'],
    '数量': [10, 15, 7, 10]
    }
    #数据类型转换：将字典类型数据转换为pandas中表格类型数据
    df = pd.DataFrame(data)
    #输出
    print("数据:")
    print(df)

    # 方法1: 使用DataFrame.plot() (推荐,简洁)
    print("\n方法1: df.plot() 绘制")
    #绘制条形图，kind表示绘制图形类型，x表示x轴用水果名称这列，y表示y轴用数量这列，title表示图形标题，
color表条形颜色，legend表示图例
    df.plot(kind='bar', x='水果', y='数量', title='水果数量分布',
    color='skyblue', legend=False)
    #设置x轴和y轴标签
    plt.xlabel('水果类型')
    plt.ylabel('数量')
    #设置x轴刻度标签不旋转
    plt.xticks(rotation=0)
    #设置自动调整布局
    plt.tight_layout()

    #将当前图表保存为图片，dpi分辨率，bbox_inches自动裁剪多余空白
    plt.savefig('条形图.png', dpi=100, bbox_inches='tight')
    #输出提示
    print("条形图已保存: 条形图.png")
    #显示图形
    plt.show()

    # 方法2: 使用plt.bar() (更灵活)
    print("\n方法2: plt.bar() 绘制")
    #创建一个新窗口
    plt.figure(figsize=(8, 5))
    #绘制条形图，调出水果列作为x轴，数量列作为y轴，color设置条形颜色，width设置条形宽度
    plt.bar(df['水果'], df['数量'], color='lightcoral', width=0.6)

    #设置图形的标题并设置标题字体大小
```

```
    plt.title('水果数量分布', fontsize=14)
    #设置x轴，y轴标题
    plt.xlabel('水果类型')
    plt.ylabel('数量')
    #设置风格线
    plt.grid(axis='y', alpha=0.3)
    #自动设整布局
    plt.tight_layout()
    #显示图形
    plt.show()

#定义绘制饼图的函数
def plot_pie_chart():
    """
     饼图 - 用于展示各部分占总体的比例
     考试常见：占比分析、百分比展示
    """
    #输出
    print("\n" + "="*60)
    print("2. 饼图绘制")
    print("="*60)

    #创建字典类型的数据
    data = {
    '水果': ['苹果', '香蕉', '樱桃', '葡萄'],
    '数量': [10, 15, 7, 10]
    }
    #数据类型转换：将字典类型数据转换为表格类型数据
    df = pd.DataFrame(data)
    #输出
    print("数据:")
    print(df)

    print("\n绘制饼图...")
    #创建一个窗口
    plt.figure(figsize=(8, 8))

    #绘制饼图，用数量这列作为扇形区大小
    plt.pie(df['数量'],
    #用水果这列作为扇形区每块标签名
    labels=df['水果'],
    #显示百分比
    autopct='%1.1f%%',
    #设置饼图从90度方向开始绘制
    startangle=90,
    #设置每个扇形区的颜色
    colors=['#ff9999', '#66b3ff', '#99ff99', '#ffcc99'],
    #设置扇形区是否突出显出
    explode=(0.1, 0, 0, 0))
    #设置图形标题和标题字体大小
    plt.title('水果数量占比', fontsize=14)
    #设置饼图显示为正圆

    plt.axis('equal')
```

```
    #自动调整布局
    plt.tight_layout()
    #将图形保存为图片，dpi设置分辨率，bbox_inches自动裁剪空白区域
    plt.savefig('饼图.png', dpi=100, bbox_inches='tight')
    print("饼图已保存：饼图.png")
    #显示图形
    plt.show()

#定义绘制拆线图的函数
def plot_line_chart():
    """
    折线图 - 用于展示数据随时间的变化趋势
    考试常见：销售趋势、成绩变化等
    """
    #输出提示
    print("\n" + "="*60)
    print("3. 折线图绘制")
    print("="*60)
    #创建字典类型数据
    data = {
    '日期': ['2025-01-01', '2025-02-01', '2025-03-01', '2025-04-01'],
    '销售额': [200, 220, 240, 260]
    }
    #数据类型转换，将字典类型数据转换为表格类型数据
    df = pd.DataFrame(data)
    #将日期这列的字符串类型转换为datetime日期时间类型
    df['日期'] = pd.to_datetime(df['日期'])
    #输出提示
    print("数据:")
    print(df)
    # 方法1：使用DataFrame.plot()
    print("\n方法1：df.plot() 绘制")
    #绘制拆线图，x表示横轴，y表示纵轴，kind表示图形类型
    df.plot(x='日期', y='销售额', kind='line',
    title='月度销售额趋势', #标题
    marker='o', # 添加数据点标记
    color='green', #拆线颜色
    legend=False) #不显示图例
    #设置x轴，y轴标签
    plt.xlabel('日期')
    plt.ylabel('销售额(元)')
    #添加风格线，alpha设置网络线的透明度
    plt.grid(True, alpha=0.3)
    #自动布局
    plt.tight_layout()
    #将图形保存为图片，dpi分辨率，bbox_inches自动裁剪空白区
    plt.savefig('折线图.png', dpi=100, bbox_inches='tight')
    #输出提示
    print("折线图已保存：折线图.png")
    #显示图形
    plt.show()


    # 方法2：使用plt.plot()
```

```
    print("\n方法2: plt.plot() 绘制")
    #创建新窗口
    plt.figure(figsize=(10, 5))
    #绘制拆线图
    plt.plot(df['日期'], df['销售额'],#日期列作为x轴，销售额这列作为y轴
    marker='o', markersize=8, #标记为圆点,圆点大小为8px
    linestyle='-', linewidth=2,#线条为实线,线宽2px
    color='blue', label='销售额') #线条颜色,线条图例名称为销售额
    #遍历销售额这一列，i索引位置，value对应当前销售额数值
    for i, value in enumerate(df['销售额']):
       #在图表中添加文字标签
        plt.text(df['日期'][i], value + 5, str(value), #x坐标,y坐标,销售额数值
        ha='center', va='bottom', fontsize=10) #水平居中,文字位于坐标点上方,字体大小10px
        #设置图形标题及标题字体大小
        plt.title('月度销售额趋势', fontsize=14)
        #设置x轴，y轴的标签
        plt.xlabel('日期')
        plt.ylabel('销售额(元)')
        #设置图例
        plt.legend()
        #设置网格线
        plt.grid(True, alpha=0.3)
        #设置x轴标签旋转15度
        plt.xticks(rotation=15)
        #自动布局
        plt.tight_layout()
         #显示图形
        plt.show()

#定义绘制综合图表的函数
def comprehensive_example():
    """
    综合示例：一次性绘制多个子图
    考试加分项：展示更高级的可视化能力
    """
    #输出提示
    print("\n" + "="*60)
    print("4. 综合示例 - 多子图布局")
    print("="*60)

   #创建字典数据
    data = {
    '科目': ['语文', '数学', '英语', '物理'],
    '成绩': [85, 92, 78, 88]
    }
    #数据类型转换
    df = pd.DataFrame(data)
    #创建一个2行2列饮食4个子图的窗口
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    #设置整张图的总标题
    fig.suptitle('学生成绩分析报告', fontsize=16, fontweight='bold')


    #绘制第1个子图，且是条形图
```

```
    axes[0, 0].bar(df['科目'], df['成绩'], color='skyblue')
    axes[0, 0].set_title('各科成绩对比')
    axes[0, 0].set_ylabel('成绩')
    axes[0, 0].grid(axis='y', alpha=0.3)#添加横向风格线

    #绘制第2个子图，且是饼图
    axes[0, 1].pie(df['成绩'], labels=df['科目'], autopct='%1.1f%%', startangle=90)
    axes[0, 1].set_title('成绩占比')

    #绘制第3个子图，且是拆线图
    axes[1, 0].plot(df['科目'], df['成绩'], marker='o', linewidth=2, color='green')
    axes[1, 0].set_title('成绩趋势')
    axes[1, 0].set_ylabel('成绩')
    axes[1, 0].grid(True, alpha=0.3)

    #绘制第4个子图，且是水平条形图
    axes[1, 1].barh(df['科目'], df['成绩'], color='coral')
    axes[1, 1].set_title('成绩排名')
    axes[1, 1].set_xlabel('成绩')
    axes[1, 1].grid(axis='x', alpha=0.3)
    #自动调整布局
    plt.tight_layout()
    #将图形保存为图片
    plt.savefig('综合分析图.png', dpi=150, bbox_inches='tight')
    print("综合分析图已保存：综合分析图.png")
    #显示图形
    plt.show()

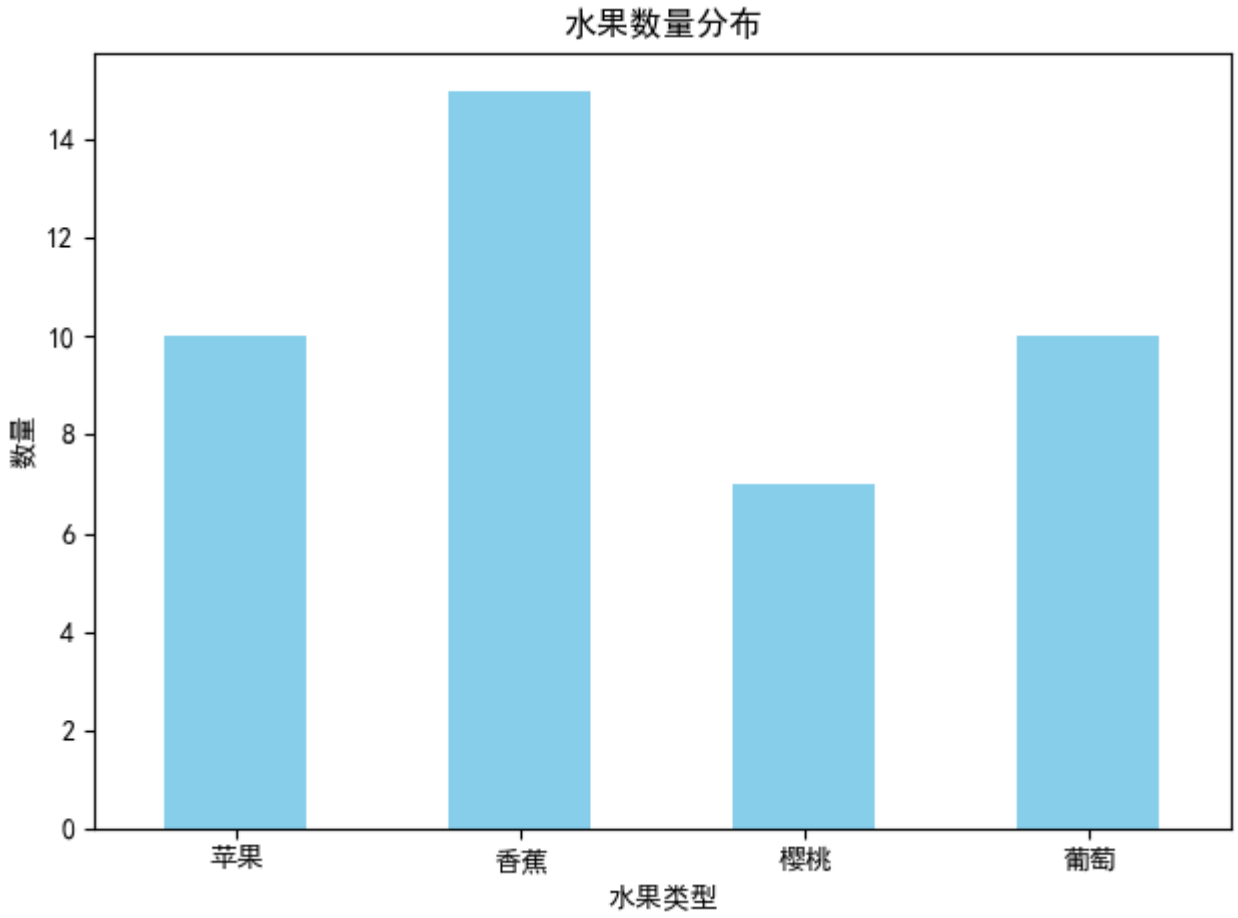
#定义主函数
def main():
    """主函数"""
    #输出提示
    print("数据可视化考点示例程序")
    print("="*60)
    #调用绘制3种基本图形的函数
    plot_bar_chart()
    plot_pie_chart()
    plot_line_chart()
    #调用绘制综合图形的函数
    comprehensive_example()
    #输出提示语
    print("\n" + "="*60)
    print("所有图表绘制完成!")
    print("="*60)

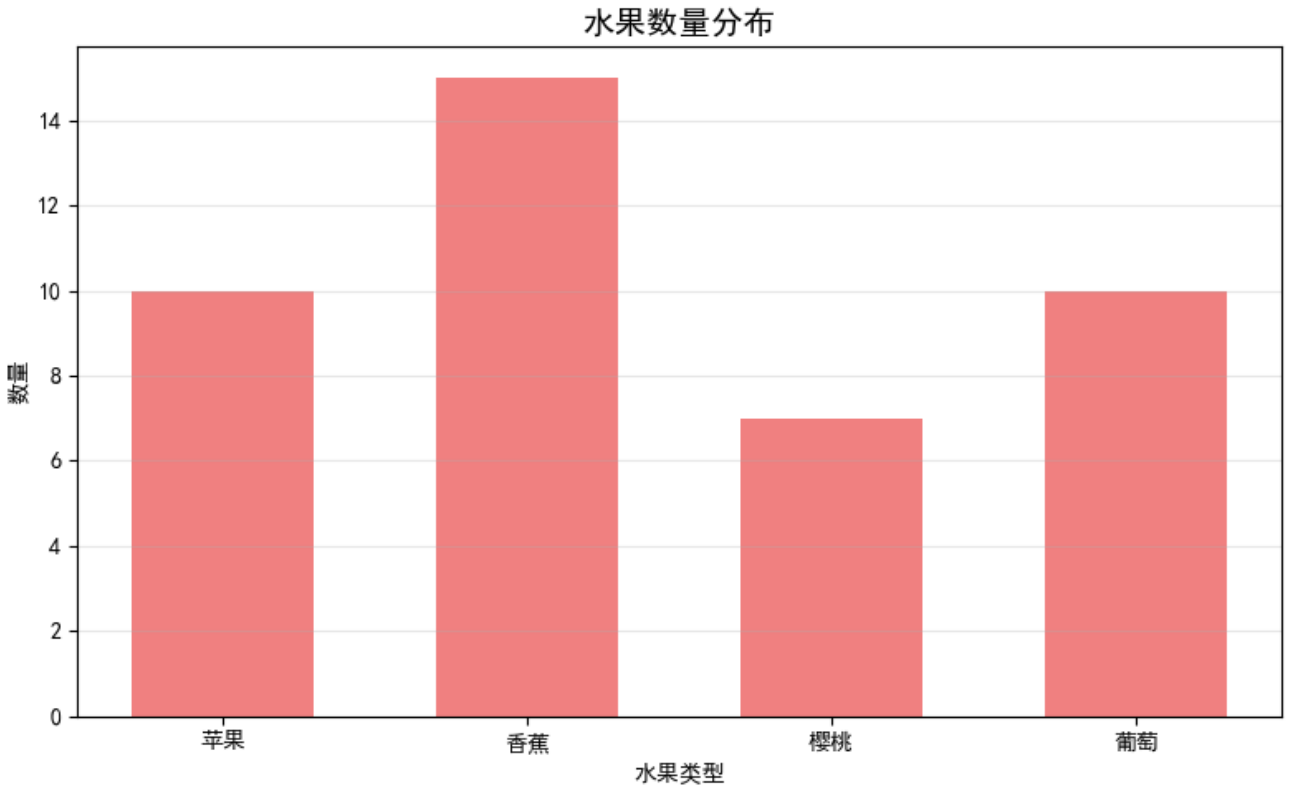
#python程序的入口
if __name__ == "__main__":
    #调用主函数
    main()
```

运行效果如下：

```
数据可视化考点示例程序
============================================================
============================================================
1. 条形图绘制
============================================================
数据:
   水果  数量
0  苹果  10
1  香蕉  15
2  樱桃   7
3  葡萄  10

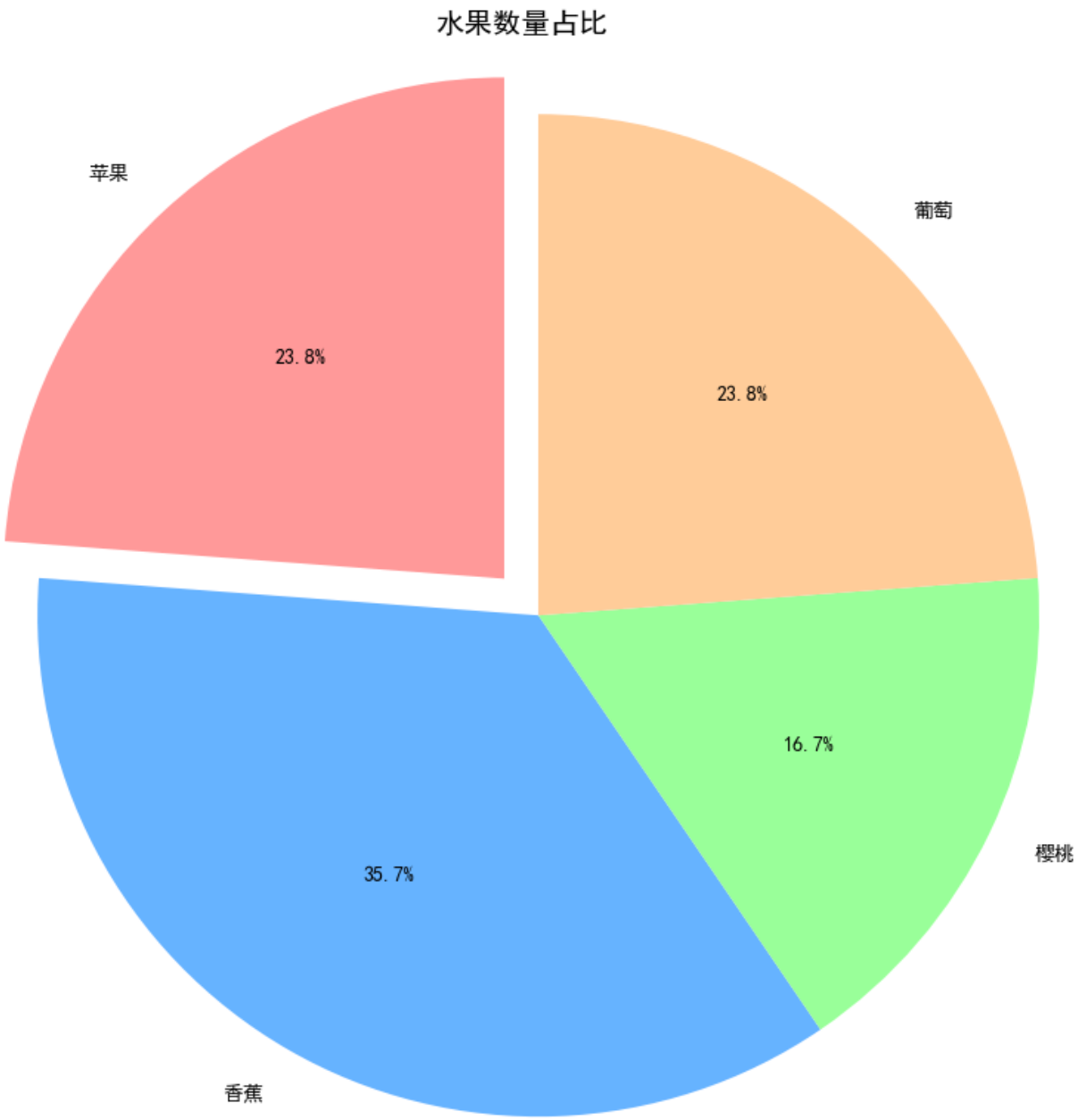
方法1: df.plot() 绘制
条形图已保存: 条形图.png
```





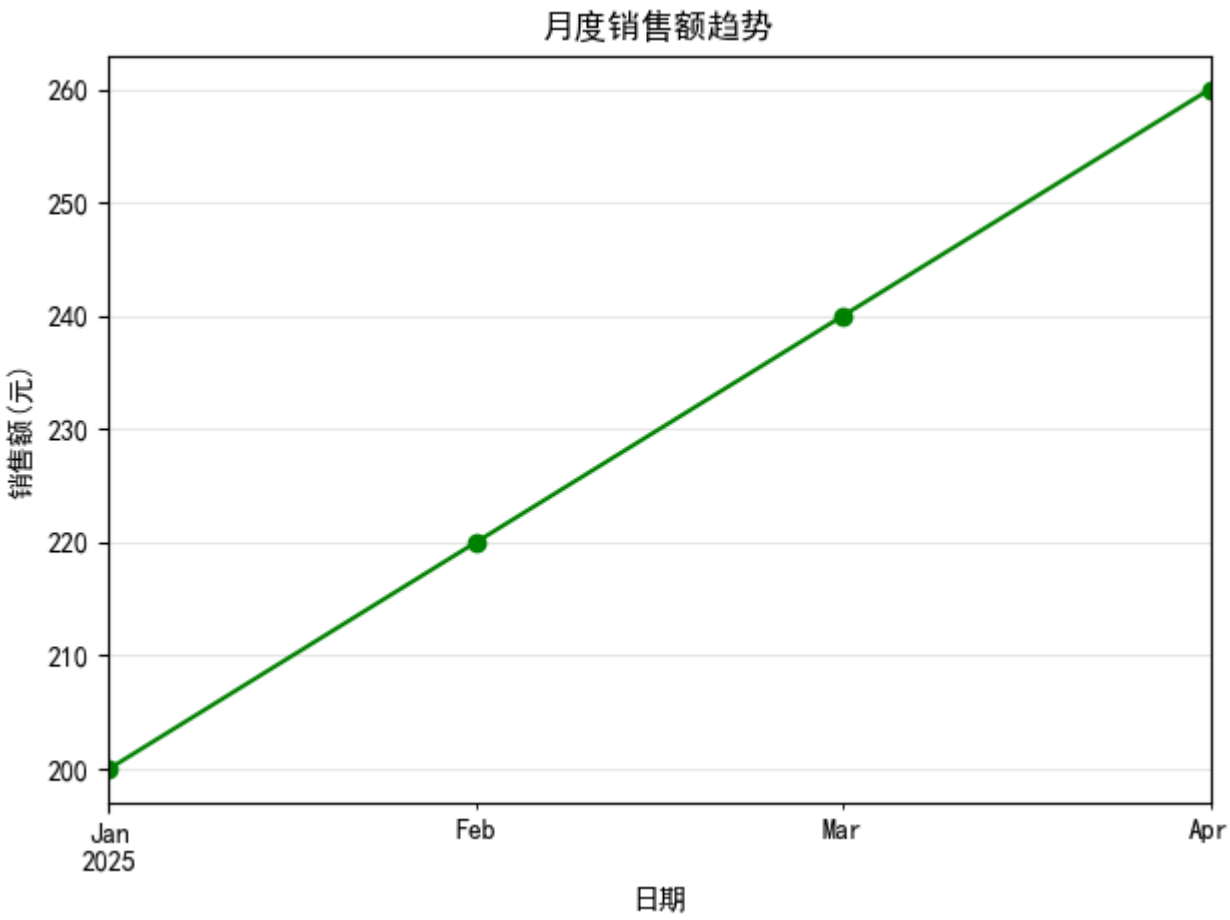

```
============================================================
2. 饼图绘制
============================================================
数据:
   水果  数量
0  苹果  10
1  香蕉  15
2  樱桃   7
3  葡萄  10

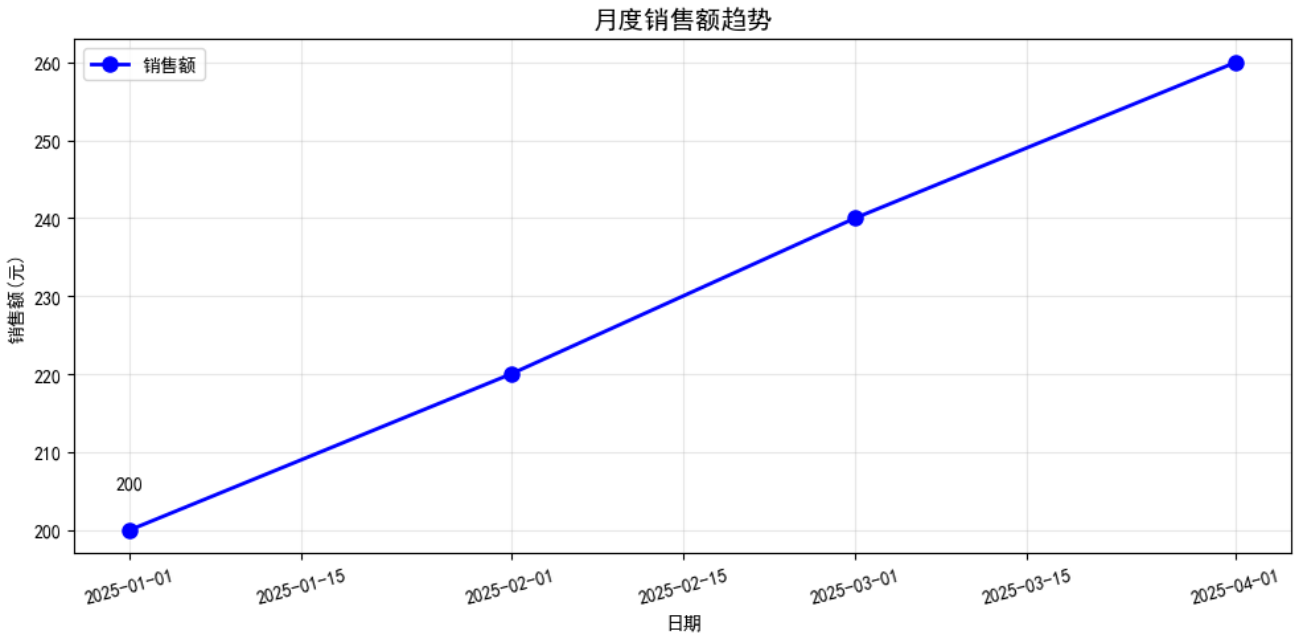
绘制饼图...
饼图已保存: 饼图.png
```



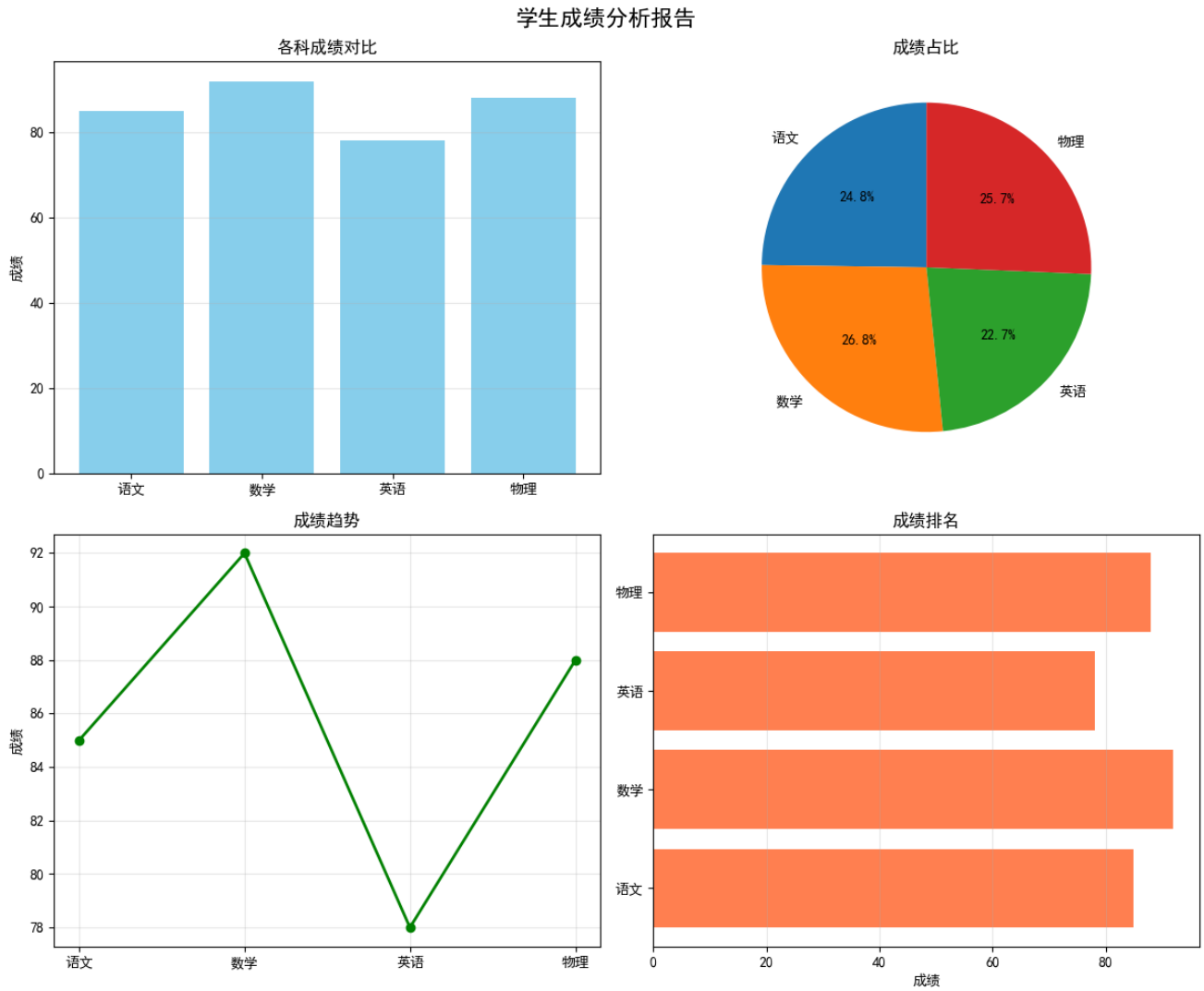

```
============================================================
3. 折线图绘制
============================================================
数据:
         日期  销售额
0 2025-01-01  200
1 2025-02-01  220
2 2025-03-01  240
3 2025-04-01  260

方法1: df.plot() 绘制
折线图已保存: 折线图.png
```






```
============================================================
4. 综合示例 - 多子图布局
============================================================
综合分析图已保存：综合分析图.png
```



2.案例二:

```
#导入读取文件数据和处理表格数据库，并取了个别名pd
import pandas as pd
#导入绘图库，并取了个别名plt
import matplotlib.pyplot as plt
#导入计算数值的库，并取了个别名np
import numpy as np

#处理中文乱码问题
#plt.rcParams['font.sans-serif'] = ['Arial Unicode MS'] # Mac
plt.rcParams['font.sans-serif'] = ['SimHei'] # Windows
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

#定义绘制条形图的函数
def plot_bar_chart():
    """
    条形图 - 用于展示类别间的数值对比
    考试常见：各科成绩对比、销售额对比等

    """
```

```
    #输出
    print("="*60)
    print("1. 条形图绘制")
    print("="*60)
    #创建一个字典数据
    data = {
    '水果': ['苹果', '香蕉', '樱桃', '葡萄'],
    '数量': [10, 15, 7, 10]
    }
    #数据类型转换
    df = pd.DataFrame(data)
    #输出提示
    print("数据:")
    print(df)
    # 方法1: 使用DataFrame.plot() (推荐,简洁)
    print("\n方法1: df.plot() 绘制")
    #绘制条形图
    df.plot(kind='bar', x='水果', y='数量', title='水果数量分布',
    color='skyblue', legend=False)
    #设置x,y轴标签
    plt.xlabel('水果类型')
    plt.ylabel('数量')
    # 横轴标签不旋转
    plt.xticks(rotation=0)
    # 自动调整布局
    plt.tight_layout()
    #将当前图形保存为图片
    plt.savefig('条形图.png', dpi=100, bbox_inches='tight')
    #输出提示
    print("条形图已保存: 条形图.png")
    #显示图形
    plt.show()
    # 方法2: 使用plt.bar() (更灵活)
    print("\n方法2: plt.bar() 绘制")
    #创建一个新窗口
    plt.figure(figsize=(8, 5))
    #绘制条形图
    plt.bar(df['水果'], df['数量'], color='lightcoral', width=0.6)
    #设置图形标题及标题的字体大小
    plt.title('水果数量分布', fontsize=14)
    #设置x,y轴标签
    plt.xlabel('水果类型')
    plt.ylabel('数量')
    # 添加网格线(加分项)
    plt.grid(axis='y', alpha=0.3)
    #自动布局
    plt.tight_layout()
    #显示图形
    plt.show()

#定义绘制饼图的函数
def plot_pie_chart():

    """
```

```
    饼图 - 用于展示各部分占总体的比例
    考试常见：占比分析、百分比展示
    """
    #输出提示
    print("\n" + "="*60)
    print("2. 饼图绘制")
    print("="*60)
    # 使用同样的数据
    data = {
    '水果': ['苹果', '香蕉', '樱桃', '葡萄'],
    '数量': [10, 15, 7, 10]
    }
    #数据类型转换
    df = pd.DataFrame(data)
    #输出提示
    print("数据:")
    print(df)
    # 绘制饼图
    print("\n绘制饼图...")
    #创建新窗口
    plt.figure(figsize=(8, 8))
    # 考点：autopct显示百分比，startangle调整起始角度
    #绘制饼图
    plt.pie(df['数量'], #用数量这列表示扇形区的大小
    labels=df['水果'], #用水果这列表示扇形区的每个区名称
    autopct='%1.1f%%', # 显示百分比,保留1位小数
    startangle=90, # 从90度开始(顶部)
    colors=['#ff9999', '#66b3ff', '#99ff99', '#ffcc99'],#设置每块扇形区的颜色
    explode=(0.1, 0, 0, 0)) # 突出第一块(可选)
    #设置图形标题
    plt.title('水果数量占比', fontsize=14)
    plt.axis('equal') # 确保饼图是圆形
    #自动布局
    plt.tight_layout()
    #将图形保存为图片
    plt.savefig('饼图.png', dpi=100, bbox_inches='tight')
    #输出提示
    print("饼图已保存：饼图.png")
    #显示图形
    plt.show()

#定义绘制拆线图的函数
def plot_line_chart():
    """
    折线图 - 用于展示数据随时间的变化趋势
    考试常见：销售趋势、成绩变化等
    """
    #输出
    print("\n" + "="*60)
    print("3. 折线图绘制")
    print("="*60)
    #准备字典数据

    data = {
```

```
'日期': ['2025-01-01', '2025-02-01', '2025-03-01', '2025-04-01'],
'销售额': [200, 220, 240, 260]
}
#数据类型转换
df = pd.DataFrame(data)
#数据类型转换，将日期这个字符串列转换为datetime日期时间类型
df['日期'] = pd.to_datetime(df['日期'])
#输出提示
print("数据:")
print(df)
# 方法1: 使用DataFrame.plot()
print("\n方法1: df.plot() 绘制")
#绘制拆线图
df.plot(x='日期', y='销售额', kind='line',#x轴显示日期列，y轴显示销售额列，kind表示图形类型
title='月度销售额趋势', #图形标题
marker='o', # 添加数据点标记
color='green', #线条颜色
legend=False) #图例
#设置x,y轴标签
plt.xlabel('日期')
plt.ylabel('销售额(元)')
#设置网络线
plt.grid(True, alpha=0.3)
#自动布局
plt.tight_layout()
#将图形保存为图片
plt.savefig('折线图.png', dpi=100, bbox_inches='tight')
#输出提示
print("折线图已保存: 折线图.png")
#显示图形
plt.show()
# 方法2: 使用plt.plot()
print("\n方法2: plt.plot() 绘制")
#创建一个新窗口
plt.figure(figsize=(10, 5))
#绘制拆线图
plt.plot(df['日期'], df['销售额'], #x轴显示日期列，y轴显示销售额
marker='o', markersize=8, #数据标记，标记大小
linestyle='-', linewidth=2, #线条类型，线条宽度
color='blue', label='销售额') #线条颜色，标签
# 考点: 添加数据标签(加分项)
#遍历销售额，i表示索引，value当前遍历销售额数值
for i, value in enumerate(df['销售额']):
#设置数据点文本标签
plt.text(df['日期'][i], value + 5, str(value),
ha='center', va='bottom', fontsize=10)
#设置标题及标题的字体大小
plt.title('月度销售额趋势', fontsize=14)
#设置x,y轴标签
plt.xlabel('日期')
plt.ylabel('销售额(元)')
#设置图例

plt.legend()
```

```
    #设置风格线
    plt.grid(True, alpha=0.3)
    # 横轴标签旋转15度
    plt.xticks(rotation=15)
    #自动调整布局
    plt.tight_layout()
    #显示图形
    plt.show()

#定义绘制综合图形的函数
def comprehensive_example():
    """
    综合示例：一次性绘制多个子图
    考试加分项：展示更高级的可视化能力
    """
    #输出提示
    print("\n" + "="*60)
    print("4. 综合示例 - 多子图布局")
    print("="*60)
    #字典数据
    data = {
    '科目': ['语文', '数学', '英语', '物理'],
    '成绩': [85, 92, 78, 88]
    }
    #数据类型转换
    df = pd.DataFrame(data)
    #绘制2行2列包含4个子图的新窗口，axes表示图，fig表示整个窗体
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    #设置整个图形标题，标题字体大小，标题字体粗细
    fig.suptitle('学生成绩分析报告', fontsize=16, fontweight='bold')
    #第一个子图，且是条形图
    axes[0, 0].bar(df['科目'], df['成绩'], color='skyblue')
    axes[0, 0].set_title('各科成绩对比')
    axes[0, 0].set_ylabel('成绩')
    axes[0, 0].grid(axis='y', alpha=0.3)
    #第二个子图，且是饼图
    axes[0, 1].pie(df['成绩'], labels=df['科目'], autopct='%1.1f%%', startangle=90)
    axes[0, 1].set_title('成绩占比')
    #第三个子图，且是拆线图
    axes[1, 0].plot(df['科目'], df['成绩'], marker='o', linewidth=2, color='green')
    axes[1, 0].set_title('成绩趋势')
    axes[1, 0].set_ylabel('成绩')
    axes[1, 0].grid(True, alpha=0.3)
    3.案例三：
    #第四个子图，且是水平条形图
    axes[1, 1].barh(df['科目'], df['成绩'], color='coral')
    axes[1, 1].set_title('成绩排名')
    axes[1, 1].set_xlabel('成绩')
    axes[1, 1].grid(axis='x', alpha=0.3)
    #自动布局
    plt.tight_layout()
    #将图形保存为图片

    plt.savefig('综合分析图.png', dpi=150, bbox_inches='tight')
```

```
    #输出提示
    print("综合分析图已保存：综合分析图.png")
    #显示图形
    plt.show()

#定义主函数
def main():
    """主函数"""
    #输出提示
    print("数据可视化考点示例程序")
    print("="*60)
    #调用三种绘制基本图形的函数
    plot_bar_chart()
    plot_pie_chart()
    plot_line_chart()
    #调用绘制综合图形的函数
    comprehensive_example()
    #输出提示
    print("\n" + "="*60)
    print("所有图表绘制完成!")
    print("="*60)

#python程序入口
if __name__ == "__main__":
    #调用主函数
    main()
```

3.案例三:

```
# 考点1：导入数据分析与可视化库
#导入pandas库，用于读取csv文件和处理表格数据，并取了个别名pd
import pandas as pd
#导入绘图库，并取了个别名plt
import matplotlib.pyplot as plt

#处理中文乱码问题
#plt.rcParams['font.sans-serif'] = ['Arial Unicode MS'] # Mac
plt.rcParams['font.sans-serif'] = ['SimHei'] # Windows
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 考点2：读取本地CSV文件（路径替换为你的实际路径）
#声明变量存csv文件路径
file_path = r"C:\Users\zjhlqc\Desktop\人工智能训练师\深技师三级实操\水果.csv"
# Pandas读取CSV为DataFrame
df = pd.read_csv(file_path)

# 预览数据（验证读取结果）
#输出
print("数据表内容：")
print(df)
```

```
# 考点3：绘制条形图（对比不同水果数量）
#创建新窗口
plt.figure(figsize=(8, 5))
#绘制条形图，df['类型']表示x轴，df['数量']表示y轴，color表示每条形颜色
plt.bar(df['类型'], df['数量'], color=['#FF7F50', '#FFD700', '#FFA500', '#FF6347'])
#设置图形标题及标题字体大小
plt.title('不同水果的数量对比', fontsize=12)
#设置x,y轴标签
plt.xlabel('水果类型', fontsize=10)
plt.ylabel('数量', fontsize=10)
#遍历数量列，i表示索引位置，v表示遍历的每个数量值
for i, v in enumerate(df['数量']):
    #设置在每个条形上方显示数量文字
    plt.text(i, v + 5, str(v), ha='center')
#自动调整布局
plt.tight_layout()
#显示图形
plt.show()

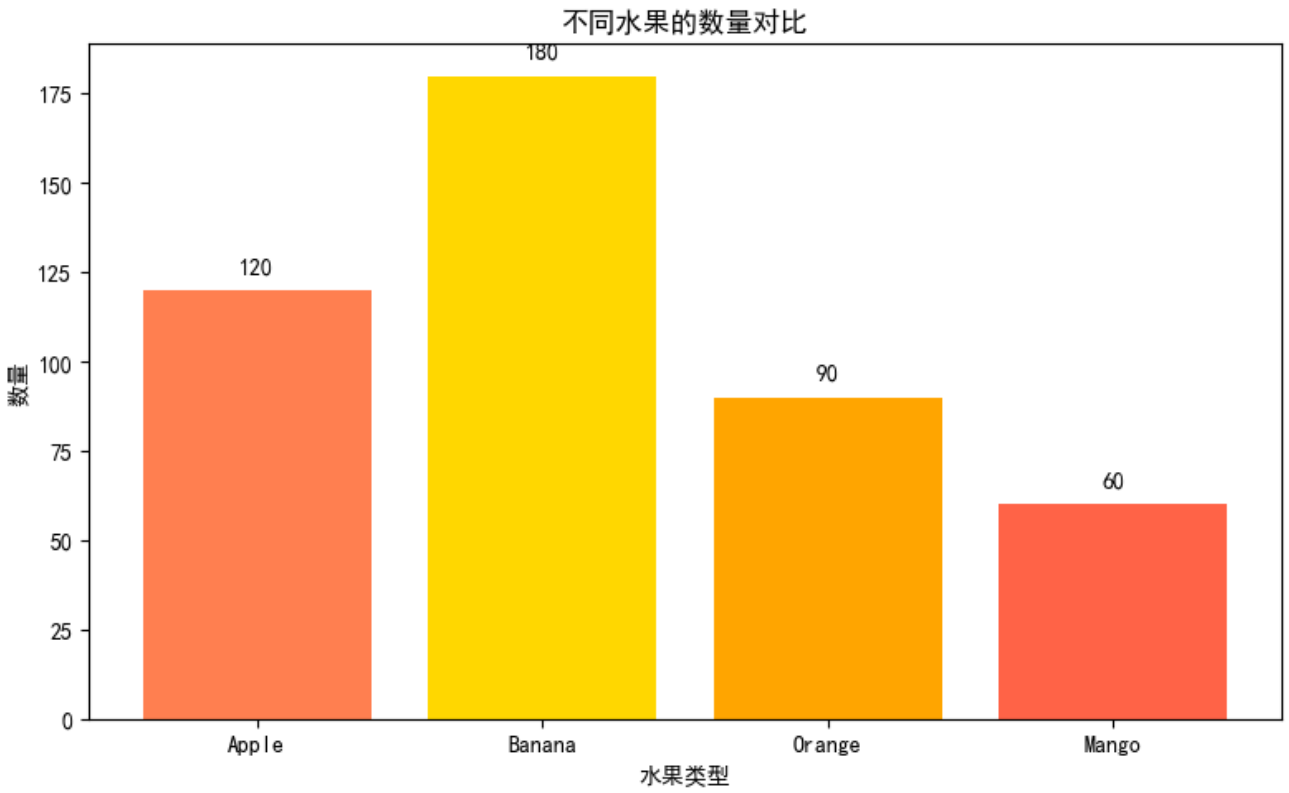

# 考点4：绘制饼图（展示水果数量占比）
#创建新窗口
plt.figure(figsize=(6, 6))
#绘制饼图
plt.pie(
    df['数量'],  #用数量列表示饼图中每一块扇形的大小
    labels=df['类型'],#用类型列设置每块扇形的标签
    autopct='%1.1f%%',  # 百分比格式（保留1位小数）
    colors=['#FF7F50', '#FFD700', '#FFA500', '#FF6347'],#设置每块扇形的颜色
    startangle=90 #设置饼图从90度开始绘制
)
#设置图形的标题和标题的字体大小
plt.title('各水果数量占比', fontsize=12)
# 保证饼图为正圆
plt.axis('equal')
#自动调整布局
plt.tight_layout()
#图像显示
plt.show()

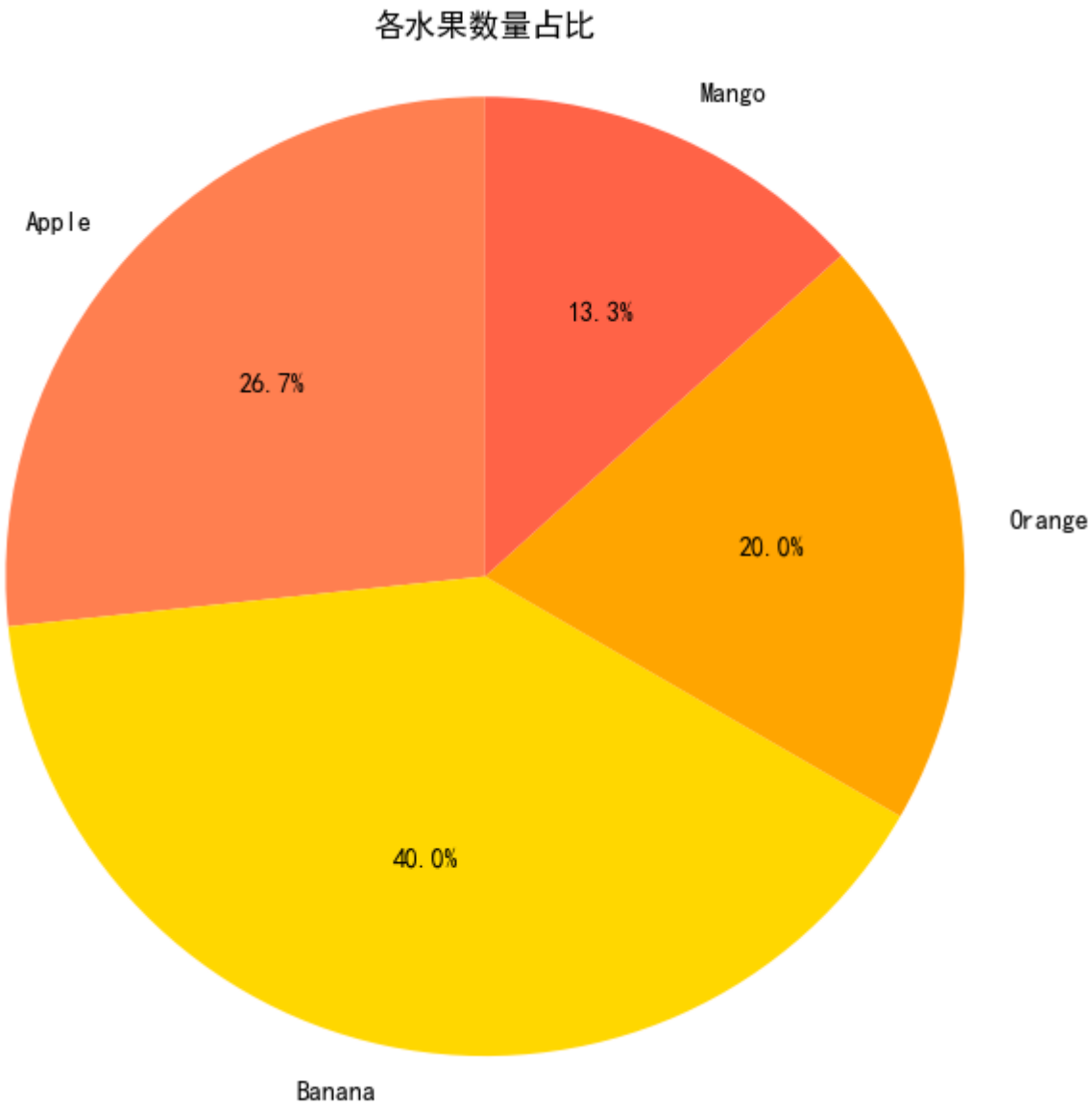

# 考点5：绘制折线图（假设补充“季度销量”数据）
#创建一个新DataFrame存在变量df_line
df_line = pd.DataFrame({
    '季度': ['Q1', 'Q2', 'Q3', 'Q4'],
    'Apple销量': [110, 120, 100, 130],
    'Banana销量': [170, 180, 160, 190]
})
#创建新窗口
plt.figure(figsize=(8, 5))
#绘制Apple销量拆线

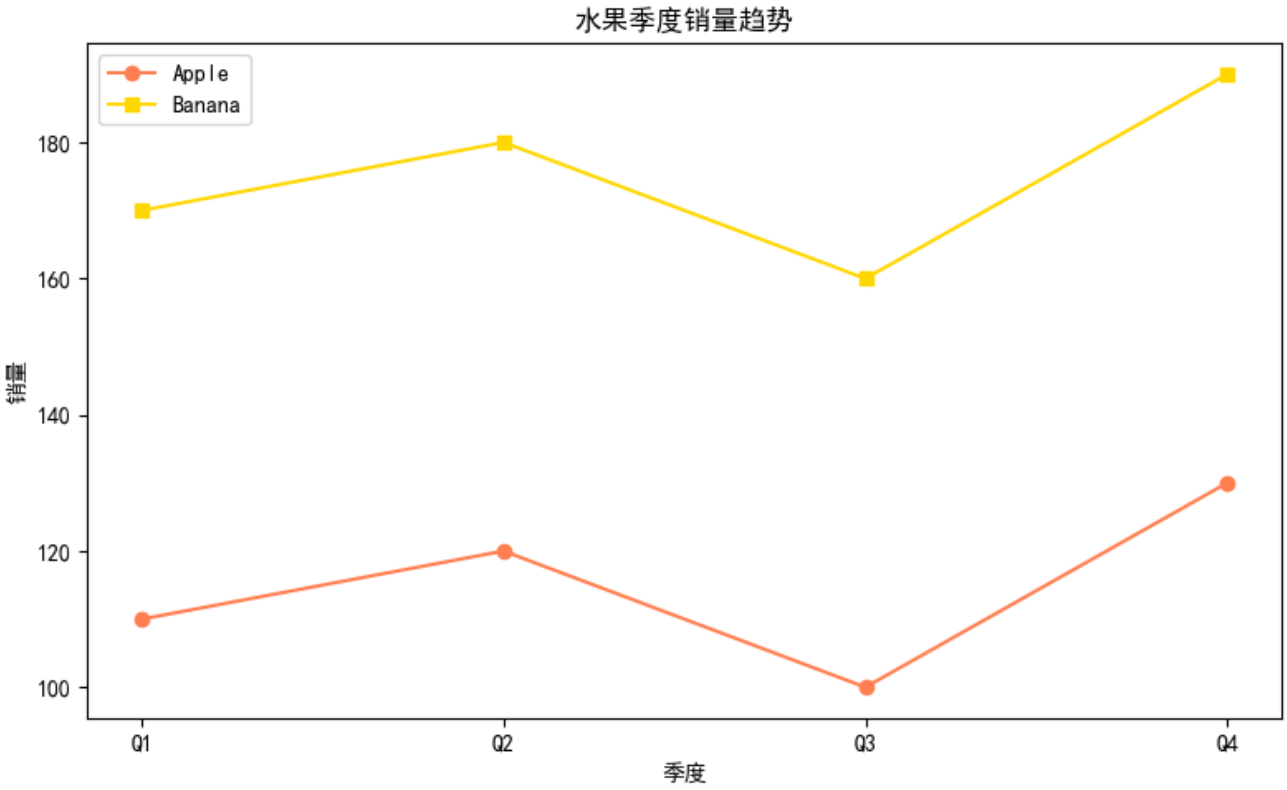
plt.plot(df_line['季度'], df_line['Apple销量'], marker='o', label='Apple', color='#FF7F50')
```

```
#绘制Banana销量拆线
plt.plot(df_line['季度'], df_line['Banana销量'], marker='s', label='Banana', color='#FFD700')
#设置拆线图标题和标题字体大小
plt.title('水果季度销量趋势', fontsize=12)
#设置x,y轴标签
plt.xlabel('季度', fontsize=10)
plt.ylabel('销量', fontsize=10)
#设置图例
plt.legend()
#自动调整布局
plt.tight_layout()
#显示图形
plt.show()
```

案例三运行效果:




各水果数量占比
Mango
13.3%
Orange
20.0%
Banana
40.0%
Apple
26.7%



四.数据分析模块（图表可视化）操作流程（含题目要求 + 代码简析）

1.题目要求分析

此模块的任务是使用表格数据绘制条形图（用于数量对比）、饼图（用于展示占比）和折线图（用于呈现趋势），并且要支持中文显示。这需要我们掌握数据处理和图表绘制的相关库和方法。

2.操作流程分析

1. **新建文件**：在 VS Code 中新建一个名为data_visualization.py的文件，用于编写绘制图表的代码。
2. **复制代码并运行**：将提供的代码复制到文件中，直接运行代码，程序会自动弹出 3 个图表窗口，分别显示条形图、饼图和折线图。

3.代码简析

导入pandas库（创建表格数据）和matplotlib.pyplot库（绘制图表）

import pandas as pd

import matplotlib.pyplot as plt

---------- 绘制条形图（水果数量对比）----------

1. 创建表格数据：水果类型和对应数量

data = {'水果': ['苹果', '香蕉', '樱桃', '葡萄'], '数量': [10, 15, 7, 10]}

转为pandas数据表

df = pd.DataFrame(data)

2. 解决中文显示乱码问题（必加，否则中文标题/标签会变成方框）

```
plt.rcParams["font.sans-serif"] = ["SimHei"] # 设置字体为黑体
plt.rcParams["axes.unicode_minus"] = False # 解决负号显示异常
# 3. 绘制条形图：x=横轴（水果类型），y=纵轴（数量），kind='bar'表示条形图
df.plot(kind='bar', x='水果', y='数量', title="水果数量分布") # title=图表标题
plt.xlabel("水果类型") # 设置横轴标签
plt.ylabel("数量（个）") # 设置纵轴标签
plt.show() # 显示条形图（弹出窗口）
# ---------- 绘制饼图（水果数量占比）----------
# 1. 沿用上面的水果数据表格
# 2. 绘制饼图：x=数量（决定各部分大小），labels=水果类型（各部分标签）
plt.pie(df['数量'], labels=df['水果'], autopct='%1.1f%%') # autopct显示占比（保留1位小数）
plt.title('水果数量占比') # 图表标题
plt.axis("equal") # 保证饼图是正圆形（否则会变成椭圆）
plt.show() # 显示饼图
# ---------- 绘制折线图（月度销售额趋势）----------
# 1. 创建时间序列数据：日期和对应销售额
time_data = {
'日期': ['2025-01-01', '2025-02-01', '2025-03-01', '2025-04-01'],
'销售额': [200, 220, 240, 260]
}
# 转为数据表
time_df = pd.DataFrame(time_data)
# 将“日期”列转为日期格式（否则横轴会按字符串排序，影响趋势展示）
time_df['日期'] = pd.to_datetime(time_df['日期'])
# 2. 绘制折线图：x=日期，y=销售额，marker='o'表示数据点用圆点标记
plt.plot(time_df['日期'], time_df['销售额'], marker='o')
plt.title('月度销售额趋势') # 图表标题
plt.xlabel('日期') # 横轴标签
plt.ylabel('销售额（元）') # 纵轴标签
plt.grid(alpha=0.3) # 添加网格线（alpha=透明度，0.3不影响美观）
plt.show() # 显示折线图
```

- **导入库**：pandas用于创建和处理表格数据，matplotlib.pyplot用于绘制图表。

- **绘制条形图**:
-
  - 创建水果数量数据的DataFrame。
  - 设置字体和负号显示，解决中文乱码问题。
  - 使用df.plot方法绘制条形图，设置横轴、纵轴和标题，添加轴标签，最后显示图表。
- **绘制饼图**:
-
  - 沿用水果数据。
  - 使用plt.pie方法绘制饼图，设置数据、标签和占比显示格式，设置标题和保证饼图为正圆形，最后显示图表。
- **绘制折线图**:
-
  - 创建时间序+列数据的DataFrame，将“日期”列转换为日期格式。
  - 使用plt.plot方法绘制折线图，设置数据点标记，添加标题和轴标签，添加网格线，最后显示图表。

五.考试要点总结(数据可视化):

1.必背代码模板:

1. 条形图:
   df.plot(kind='bar', x='类别列', y='数值列', title='标题')
   plt.show()
2. 饼图:
   plt.pie(df['数值列'], labels=df['标签列'], autopct='%1.1f%%')
   plt.title('标题')
   plt.show()
3. 折线图:
   df['日期'] = pd.to_datetime(df['日期']) # 日期转换
   df.plot(x='日期', y='数值', kind='line', marker='o')
   plt.show()

2.必须掌握的函数:

1. pd.DataFrame(data) # 数据转换
2. df.plot() # DataFrame绘图
3. plt.pie() # 饼图
4. plt.show() # 显示图表
5. plt.title() # 标题
6. plt.xlabel() / plt.ylabel()# 轴标签
7. plt.savefig() # 保存图片
8. pd.to_datetime() # 日期转换

3.易错点:

1. 忘记 plt.show() 导致图表不显示
2. 中文显示乱码(需设置字体)
3. 饼图忘记 autopct='%1.1f%%' 不显示百分比
4. 折线图日期列忘记用 pd.to_datetime() 转换
5. 图表布局混乱(建议加 plt.tight_layout())

4.加分项:

1. 添加网格线: plt.grid(True, alpha=0.3)
2. 添加数据标签: plt.text()
3. 自定义颜色: color='颜色'
4. 调整图表大小: plt.figure(figsize=(宽, 高))
5. 保存高分辨率图片: plt.savefig('文件名', dpi=150)

变种题型:

- 水平条形图: kind='barh'
- 堆叠条形图: stacked=True
- 多条折线: 多次调用 plt.plot()
- 子图布局: plt.subplots(行, 列)